

1. Squared Sums

Consider the following code introduced in lecture #07:

```
1 #include <stdlib.h>
2
3 double sqsum(double *restrict a, int n) {
4     double s = 0.0;
5     for (int i = 0; i < n; i++)
6         s += a[i] * a[i];
7     return s;
8 }
9
10 double sum;
11
12 int main() {
13     int n = 8192;
14     double *a = (double *)calloc(n, sizeof(double));
15     for (int i = 0; i < 10000; i++)
16         sum = sqsum(a, n);
17 }
```

Address the following questions assuming loop unrolling with dependency break as introduced in the lecture:

- For a system of your choice, what is the optimal unrolling factor?
- Could it be better to use fused-multiply-add instructions instead of multiply and add instructions? Why is this the case and what are the relevant microarchitecture parameters?

Provide code and documentation on the used system.

2. Auto-Vectorization

Expand upon your previous implementation of the multiplication of a dense matrix and a vector:

$$\vec{y} \leftarrow A \cdot \vec{x}, \quad (1)$$

where A is a square matrix of size N and $\vec{x}$, $\vec{y}$ are vectors of length N . The elements of the matrix and both vectors are double-precision floating-point numbers.

Perform the following tasks:

- Check whether compiler performed auto-vectorization based on compiler reports and assembler inspection,
- Code changes that are necessary to facilitate auto-vectorization should be documented.